

COMPUTE, STORAGE, NETWORKING · CONTAINERS AND CLOUD RUN

Courier checkpoint

Problem statement

Two hours · individual · your own sandbox project · Cloud Shell only

Contents

The brief	3
Rules and environment	4
Task 1 — Write both Dockerfiles	5
Task 2 — Prove the container contract locally	6
Task 3 — Artifact Registry and Cloud Build	7
Task 4 — Deploy parcel-api	8
Task 5 — Deploy track-web and connect the two	9
Task 6 — Break it, fix it	10
Task 7 — Canary the ETA feature	11
Stretch task	12
Submission and teardown	13

The brief

Courier is a small parcel-tracking product. Its engineering team has written two Python services and handed them to you, the platform engineer, with one instruction: get them running on Google Cloud the way we taught you — properly packaged, privately connected, and released without downtime.

Service	What it does	Who may call it
parcel-api	JSON API: /parcels, /parcels/<id>, /healthz, /whoami, and / (service banner). Reads two environment variables: ETA_ENABLED and DEPOT.	Only track-web. Nobody on the internet.
track-web	Public web page. Looks up one parcel by id by calling parcel-api with its own identity. Reads API_URL.	Anyone.

Both services are supplied complete in the starter kit and work as they are. You will not change a line of Python. Everything you produce is packaging (two Dockerfiles), infrastructure (a registry, two services, two identities, one IAM binding) and operations (a canary release). That is exactly the split this programme has taught: the code is the developer's; how it runs is yours.

What you hand in

- SUBMISSION.md from the kit, every blank filled with the real value or command output from your project.
- Both Dockerfiles (they stay in the kit folder; the instructor reads them from your Cloud Shell or you paste them into the submission).
- Both services running in your project until the instructor has verified them — then you tear down.

Rules and environment

Rules

- Individual work in your own sandbox project. Talking is fine; sharing commands or Dockerfiles is not.
- Cloud Shell and the console only. No local installs, no code changes to app.py, gunicorn.conf.py or requirements.txt.
- The documentation is open book: docs.cloud.google.com, docs.docker.com, the two session lab manuals and your own notes.
- Time-box: 2 hours from the moment the kit is released. Submit what you have at the end; partial work scores partial points.
- Names are fixed so the instructor can verify: region asia-south1, repository courier, services parcel-api and track-web, service accounts parcel-api-sa and track-web-sa, image tags 1.0.

Environment

Item	Value
Project	Your sandbox project (Owner role), billing attached by the programme
Shell	Cloud Shell — gcloud, docker, curl pre-installed
Region	asia-south1 for everything
Base image	python:3.13-slim (pull it once early: docker pull python:3.13-slim)
Kit	gcp-checkpoint-starter.zip → courier-starter/ (parcel-api/, track-web/, SUBMISSION.md, README.md)
APIs	run, artifactregistry, cloudbuild, iam — enable them first if the project is fresh

CLOUD SHELL

```
$ unzip -q gcp-checkpoint-starter.zip && cd courier-starter && ls
README.md SUBMISSION.md parcel-api track-web
$ ls -la parcel-api track-web
parcel-api:
.  ..  .dockerignore  Dockerfile  app.py  gunicorn.conf.py  requirements.txt
track-web:
.  ..  .dockerignore  Dockerfile  app.py  requirements.txt
```

Both Dockerfiles in the kit contain only comments. That is the starting point of Task 1.

Task 1 — Write both Dockerfiles

Suggested time	Points	Produces
25 min	25	parcel-api/Dockerfile, track-web/Dockerfile

GOAL

Turn each service into a production-quality image using the five decisions from the containers session: a slim base, pinned dependencies, a multi-stage build, a non-root user, and an exec-form start command that honours the port Cloud Run injects.

REQUIREMENTS

- Base image python:3.13-slim for the runtime stage of both images.
- Dependencies installed from requirements.txt exactly as pinned — no extra packages, no unpinned installs.
- Multi-stage: pip and its cache must not be present in the runtime image.
- Only the files each service needs are copied (parcel-api: app.py and gunicorn.conf.py; track-web: app.py). No COPY . . that drags the README along.
- The container runs as a non-root user.
- parcel-api starts with gunicorn using its gunicorn.conf.py. track-web has no config file: pass the same settings on the command line (bind to \$PORT, 1 worker, 8 threads, graceful-timeout 8).
- SIGTERM must reach gunicorn so the container stops cleanly inside Cloud Run's 10-second grace period.

ACCEPTANCE CRITERIA

- docker build succeeds for both images with no warnings about the CMD form.
- Each image is under 250 MB.
- docker run --rm <image> whoami prints something other than root.
- docker history shows no pip install layer in the runtime image (the install happens in the builder stage).

ALLOWED HINTS

- The hardened Dockerfile you compared in the containers lab is the pattern. You may consult it, not paste it — the file names, the start command and the port handling differ here.
- gunicorn reads its settings from a config file OR from flags; it does not expand \$PORT inside a JSON-array CMD by itself.

Task 2 — Prove the container contract locally

Suggested time	Points	Produces
15 min	10	SUBMISSION.md sections 1–2

GOAL

Before anything reaches the cloud, show in Cloud Shell that each image honours the three things Cloud Run will demand: it listens on whatever PORT it is given, it answers a health check, and it shuts down cleanly on SIGTERM.

REQUIREMENTS

- Run parcel-api locally with PORT set to 9090 (not the default) and reach /healthz on that port.
- Run track-web locally with PORT set to 9091 and reach its /healthz.
- Stop the parcel-api container with docker stop and capture the last line it logged.
- Record both image sizes and the user the api container runs as.

ACCEPTANCE CRITERIA

- curl localhost:9090/healthz prints ok.
- docker stop returns in well under 10 seconds and the last log line reads: parcel-api: SIGTERM received, in-flight requests finished, exiting cleanly.
- SUBMISSION.md Task 1 and Task 2 sections are complete.

ALLOWED HINTS

- If docker stop takes exactly 10 seconds, the signal never reached unicorn. Look at what process is PID 1 in your container.
- The api needs no API_URL. track-web does — for the local health check any value works.

Task 3 — Artifact Registry and Cloud Build

Suggested time	Points	Produces
10 min	10	Repository + two images; SUBMISSION.md section 3

GOAL

Create the registry the team will use and get both images into it with Cloud Build, tagged as release 1.0.

REQUIREMENTS

- A Docker-format Artifact Registry repository named courier in asia-south1.
- Both images built by Cloud Build (gcloud builds submit) and pushed as <region>-docker.pkg.dev/<project>/courier/parcel-api:1.0 and .../track-web:1.0.
- The repository description mentions the checkpoint (any wording).

ACCEPTANCE CRITERIA

- gcloud artifacts docker images list on the repository shows both images with tag 1.0.
- SUBMISSION.md Task 3 section holds the repository resource name and both full image paths.

ALLOWED HINTS

- Building with docker push after gcloud auth configure-docker is also acceptable; Cloud Build is the taught path.

Task 4 — Deploy parcel-api

Suggested time	Points	Produces
15 min	15	parcel-api service; SUBMISSION.md section 4

GOAL

Put the API on Cloud Run as an internal-facing service: nobody anonymous can call it, it runs under its own least-privilege identity, and it is sized the way the brief specifies.

REQUIREMENTS

- Service name parcel-api, region asia-south1, image .../parcel-api:1.0.
- A dedicated service account parcel-api-sa with NO project roles, set as the service's runtime identity.
- Authentication required (no allow-unauthenticated).
- Memory 512Mi, 1 CPU, concurrency 80, min instances 0, max instances 3.
- Environment variable DEPOT set to BLR (the banner endpoint / must show it).

ACCEPTANCE CRITERIA

- An anonymous curl to the service URL returns HTTP 403.
- curl with your own identity token (gcloud auth print-identity-token) returns HTTP 200 and JSON with "depot":"BLR" and "eta_enabled":false.
- gcloud run services describe shows serviceAccountName parcel-api-sa@... and the sizing above.
- SUBMISSION.md Task 4 section complete.

ALLOWED HINTS

- Nothing forces you to use the command line — the console's Create service form has every one of these settings. Verify with gcloud afterwards either way.

Task 5 — Deploy track-web and connect the two

Suggested time	Points	Produces
15 min	20	track-web service, IAM binding; SUBMISSION.md section 5

GOAL

Put the public page on Cloud Run and make the two services talk: track-web calls parcel-api with its own identity, and parcel-api lets that one identity in — nothing else.

REQUIREMENTS

- Service name track-web, region asia-south1, image .../track-web:1.0, public (allow unauthenticated).
- A dedicated service account track-web-sa as its runtime identity.
- Environment variable API_URL set to the parcel-api service URL.
- An IAM binding on the parcel-api service (not on the project) granting roles/run.invoker to track-web-sa only.
- Memory 512Mi, max instances 3.

ACCEPTANCE CRITERIA

- Opening the track-web URL with ?id=CR-1002 shows recipient S. Mehta, city Pune, status out for delivery, and the banner shows parcel-api version 1.0 and depot BLR.
- gcloud run services get-iam-policy parcel-api shows exactly one invoker binding, for track-web-sa.
- parcel-api itself is still 403 to anonymous callers.
- SUBMISSION.md Task 5 section complete.

ALLOWED HINTS

- track-web mints an ID token for the api URL from the metadata server. It needs no key file and no role except invoker on the api.
- If the page says 403, the identity making the call is not the one you bound. Check which service account track-web actually runs as.

Task 6 — Break it, fix it

Suggested time	Points	Produces
5 min	5	SUBMISSION.md section 6

GOAL

Show that the IAM binding is the only thing letting track-web in — remove it, watch the page fail, put it back, watch it recover.

REQUIREMENTS

- Remove the run.invoker binding for track-web-sa from parcel-api.
- Reload the page (allow ~20 seconds for the change to propagate) and capture the error text it shows.
- Restore the binding and confirm the page works again.

ACCEPTANCE CRITERIA

- The captured error starts with could not reach parcel-api: 403.
- After restoring, ?id=CR-1002 shows the parcel again.
- SUBMISSION.md Task 6 section holds the error text and the exact restore command.

Task 7 — Canary the ETA feature

Suggested time	Points	Produces
15 min	15	Second revision, traffic split; SUBMISSION.md section 7

GOAL

The product team wants the estimated-delivery date switched on, carefully. The feature already exists in the image behind `ETA_ENABLED`. Release it as a new revision, send 20% of traffic to it, confirm it works, then promote it to 100% — with the old revision still there to roll back to.

REQUIREMENTS

- Create a new revision of `parcel-api` from the SAME image with `ETA_ENABLED=true`, without sending it any traffic at deploy time.
- Split traffic: 20% to the new revision, 80% to the previous one.
- Prove the split from the outside (loading the page for CR-1003 several times shows the estimated delivery line on roughly one load in five).
- Promote the new revision to 100% of traffic.
- Do not delete the previous revision.

ACCEPTANCE CRITERIA

- `gcloud run services describe parcel-api --format='value(status.traffic)'` showed the 20/80 split at some point and shows 100% on the newer revision at the end.
- The track-web page for CR-1003 shows estimated delivery: `<date>` on every load after promotion.
- `gcloud run revisions list` shows at least two revisions of `parcel-api`.
- SUBMISSION.md Task 7 section complete.

ALLOWED HINTS

- A configuration change is a new revision. Look for the deploy flag that keeps a new revision at zero traffic, and the `update-traffic` command from the Cloud Run session.

Stretch task

Optional, 5 bonus points, only after Tasks 1–7 are complete. Users open track-web first, so a cold start there is the one they notice. Configure track-web to keep one instance warm with startup CPU boost, then write one sentence in the submission explaining why you did this on track-web and not on parcel-api.

- gcloud run services describe track-web shows minScale 1.
- The sentence names the cost trade-off (an idle instance is billed) and the reason the public page, not the internal API, deserves it.

Submission and teardown

Submission

- Fill SUBMISSION.md in the kit — every blank, real values. Paste it into the programme's submission form, or hand it to the instructor as a file.
- Leave both services and the repository running until the instructor has verified your project (they need about five minutes per learner).
- Leave both Dockerfiles in place in Cloud Shell; the instructor reads them from there.

Teardown — after verification only

CLOUD SHELL

```
PROJECT=$(gcloud config get-value project); R=asia-south1
gcloud run services delete track-web --region $R --quiet
gcloud run services delete parcel-api --region $R --quiet
gcloud iam service-accounts delete track-web-sa@$PROJECT.iam.gserviceaccount.com --quiet
gcloud iam service-accounts delete parcel-api-sa@$PROJECT.iam.gserviceaccount.com --quiet
gcloud artifacts repositories delete courier --location=$R --quiet
docker rm -f $(docker ps -aq) 2>/dev/null; docker image prune -af
gcloud run services list --region $R      # expect: Listed 0 items.
```

CHECKLIST BEFORE YOU STOP

- Both Dockerfiles saved and building.
- SUBMISSION.md complete — no blanks left as _____.
- Both services still up for verification.
- You have noted your project ID on the submission.